

THE UNIVERSITY OF AUCKLAND

Semester One, 2025

Campus: City

Software Engineering Operating Systems

(Time Allowed: ONE hour)

NOTE:

- This exam is closed book.
- ONE double-sided A4 cheat sheet is allowed.
- The use of calculators or other devices is NOT permitted.
- Compare the exam version number on the Teleform sheet supplied with the version number above. If they do not match, ask the supervisor for a new sheet.
- Enter your name and student ID on the Teleform sheet. Your name should be entered left aligned. If your name is longer than the number of boxes provided, truncate it.
- Answer ALL multiple-choice questions on the Teleform answer sheet provided.
- Use a dark pencil to mark your answers in the multiple-choice answer boxes on the Teleform sheet. Check that the question number on the sheet corresponds to the question number in this question book. If you spoil your sheet, ask the supervisor for a replacement.
- Choose the BEST answer for each question based on the material covered in the course.
- There are 30 questions in this exam, worth 30 marks in total. Allocate your time appropriately.

Surname:

First Name(s):

Student ID:

Question 1:

When a user program calls the pipe system call in xv6, what is the correct sequence of operations that the operating system performs?

- A. The process switches to kernel mode via a trap, the syscall number is identified, and the `sys_pipe` function is called to create a pipe and return two file descriptors.
- B. The user program sends a signal to the kernel, which asynchronously allocates the pipe during the next timer interrupt.
- C. The user program directly allocates a pipe buffer in user space and returns two pointers.
- D. The kernel preempts the calling process and puts it to sleep until both ends of the pipe are opened.
- E. The process modifies the page table to map a shared memory region as a pipe buffer between processes without entering kernel mode.

Question 2:

Your system starts with a single process. After executing the code below, how many processes will exist? Assume all system calls succeed and none of the created processes have exited.

```
int main()
{
    ...
    fork();
    execl("/bin/echo", "echo", "2", NULL);
    fork();
    ...
}
```

- A. 4
- B. 2
- C. 3
- D. 1
- E. 5

Question 3:

You're implementing a user-level sleep program (`sleep [seconds]`) in xv6, but you cannot use the built-in `sleep()` system call. Instead, you choose to utilize the `uptime()` system call, which returns the number of ticks (approximately 0.1 seconds per tick) elapsed since system boot. Your implementation looks like the code snippet below. Which of the following statements best describes this implementation?

```
int main(int argc, char *argv[]) {
    int seconds = atoi(argv[1]);
    int ticks_to_wait = seconds * 10;
    int start_ticks = uptime();
    while (uptime() - start_ticks < ticks_to_wait);
    exit(0);
}
```

- A. This implementation doesn't sleep at all because the `uptime()` function doesn't increment while the process is busy-waiting.
- B. This implementation correctly sleeps the requested duration but uses significantly more CPU time.
- C. This implementation creates zombie processes due to incorrect handling of process termination.

- D. This implementation behaves exactly like the built-in sleep() and efficiently blocks the process without using CPU resources.
- E. This implementation will fail completely because uptime() cannot reliably measure small durations of time.

Question 4:

You are working to improve your TLB performance. You managed to double the TLB hit rate from 25% to 50%, but TLB access time increased from negligible to equal to memory access time. The system uses a 2-level page table. What is the improvement in expected memory access time (EAT)? (% improvement = (Old EAT – New EAT) / Old EAT)

- A. 0
- B. -10%
- C. 10%
- D. 20%
- E. -20%

Question 5:

What happens to the child process immediately after it calls exit(0)?

```
int main() {  
    if (fork() == 0) {  
        exit(0);  
    } else {  
        sleep(2);  
        wait(NULL);  
    }  
    return 0;  
}
```

- A. The child process becomes an orphan process.
- B. The child process continues running, waiting until the parent wakes up.
- C. Both the parent and child process are blocked during the 2 seconds of sleep.
- D. The child process immediately terminates and disappears from the process list.
- E. The child process becomes a zombie process until the parent wakes and calls wait().

Question 6:

How many of the following statements are TRUE? (1) On x86, a memory reference that results in a TLB miss does not always result in a page fault. (2) Memory load and store instructions do not always trap into the OS kernel. (3) On x86, the OS decides which pages to replace. (4) Compared to a single-level page table, a multi-level page table does not inherently change the TLB miss rate.

- A. 1
- B. 0
- C. 2
- D. 3
- E. 4

Question 7:

In a Unix Shell environment, the `exec()` system call is typically invoked in which process, and what part of the memory does it replace?

- A. Parent process and replaces the kernel memory space
- B. Parent process and replaces the child process's memory space
- C. Child process and replaces its own memory space
- D. Child process and replaces the parent process's memory space
- E. Parent process and replaces its own memory space

Question 8:

What is the exact output of the code snippet below?

```
int main() {  
    printf("Hello");  
    write(STDOUT_FILENO, "World", 5);  
    printf("Again");  
    return 0;  
}
```

- A. AgainHelloWorld
- B. WorldHelloAgain
- C. HelloAgainWorld
- D. WorldAgainHello
- E. HelloWorldAgain

Question 9:

You are designing a real-time application on a system with limited physical memory. After experimenting with several page replacement algorithms, you observed the following:

FIFO results in poor performance due to a high number of page faults.

LRU provides better performance but introduces significant overhead from tracking usage history.

The Clock algorithm yields fewer page faults than FIFO and offers acceptable performance, though it still falls short of expectations.

Given the system constraints (real-time processing, moderate fault tolerance, and low overhead), which optimization would you choose to improve performance most effectively?

- A. Replace Clock with Optimal replacement since it guarantees the least page faults.
- B. Use the Clock algorithm but tune the reference bit reset frequency.
- C. Switch to LRU, but implement it exactly to minimize page faults at all costs.
- D. Increase physical memory size by 1 page to 5 pages to reduce page faults under FIFO.
- E. Replace Clock with Worst Fit allocation to delay memory saturation.

Question 10:

Consider the following code, what is the output of the printf() statement?

```
struct alnt_st{
    char a;
    int b;
    char c;
    int d;
};

int main() {
    struct alnt_st test;
    printf("%d\n", sizeof(test));
    return 0;
}
```

- A. 10
- B. 16
- C. 12
- D. 9
- E. 14

Question 11:

A system is allocating memory using various allocation strategies. You have a memory pool with free blocks of sizes [5KB, 10KB, 20KB, 25KB]. A series of allocation requests arrive in this order: 8KB, 4KB, 6KB, 15KB. Given this scenario, which memory allocation strategy is most likely to cause excessive fragmentation over time but may initially result in faster allocations?

- A. Buddy Allocator
- B. Best Fit
- C. Slab Allocator
- D. Worst Fit
- E. First Fit

Question 12:

What is the exact output of the code snippet below (ignoring any new line)?

```
int main() {
    execl("/bin/echo", "echo", "2", NULL);
    printf("3");
    return 0;
}
```

- A. 32
- B. Nothing is printed
- C. 3
- D. 2
- E. 23

Question 13:

What is the exact output of the code snippet below?

```
int main() {  
    int x = 5;  
    if (fork() == 0) {  
        x += 5;  
        printf("%d ", x);  
    } else {  
        x -= 2;  
        wait(NULL);  
        printf("%d ", x);  
    }  
    printf("%d ", x);  
    return 0;  
}
```

- A. 10 10 3
- B. 10 10 3 3
- C. Output order is not deterministic
- D. 3 10 3
- E. 10 3 3

Question 14:

What is the page number of the 24-bit address 0x789ABC with a 128-byte page size?

- A. 0xBC
- B. None of the given options
- C. 0x9A
- D. 0x78
- E. 0x89A

Question 15:

True or false? (1) A context switch must happen in kernel mode. (2) A context switch can occur only after processing an interrupt. (3) A system call made by a user process always results in a transition from user space to kernel space.

- A. (1) true (2) true (3) true
- B. (1) false (2) false (3) false
- C. (1) true (2) false (3) false
- D. (1) true (2) false (3) true
- E. (1) false (2) false (3) true

Question 16:

True or false? On a copy-on-write system, after calling fork(): (1) Parent and child share a page translation table. (2) Parent and child share virtual memory. (3) Parent and child share physical memory.

- A. (1) true, (2) true, (3) false
- B. (1) true, (2) false, (3) true
- C. (1) false, (2) true, (3) true
- D. (1) false, (2) false, (3) true
- E. (1) false, (2) false, (3) false

Question 17:

The TLB can be accessed in 5 clock cycle, and memory access takes 75 clock cycles. A page table entry can be found in 100 clock cycles. Assuming there are no page faults, what is the minimum TLB hit percentage required to keep the expected clock cycles at most 90?

- A. 0.95
- B. 0.85
- C. 0.9
- D. 0.8
- E. 0.75

Question 18–20:

Consider a system with a 25-bit virtual address, a 40-bit physical address, a 128-byte page size, and a PTE size of 1 byte. For a multi-level page table, assuming each table fits within a physical frame.

Question 18:

For a single-level page table, how many PTEs are there?

- A. 2^{21}
- B. 2^{17}
- C. 2^{19}
- D. 2^{18}
- E. 2^{20}

Question 19:

For a multi-level page table, what is the minimum number of page table levels required?

- A. 5
- B. 4
- C. 2
- D. 1
- E. 3

Question 20:

Your program uses 130 pages for data (including code). For a multi-level page table, what is the minimum possible number of pages that your program could require?

- A. 132
- B. 134
- C. 131
- D. 133
- E. 130

Question 21:

Assume our physical memory can only hold 3 pages, and we access the following: 1 2 3 1 2 4 1 2 3 (all the pages are initially on disk). What are the number of page faults for Optimal, FIFO, LRU?

- A. 5, 7, 6
- B. 5, 6, 6
- C. 6, 7, 6
- D. 5, 7, 5
- E. 6, 7, 7

Question 22:

Why are user mode and kernel mode separated in an operating system?

- A. All of the given options.
- B. To prevent user applications from executing privileged instructions.
- C. So the operating system cannot access user application data.
- D. To isolate user programs from critical system operations.
- E. It simplifies multitasking by allowing direct access to hardware for all applications.

Question 23–27:

There are 4 processes in your system. Only 1 process can run at a time.

P1, arrival time = 0, burst time = 4

P2, arrival time = 1, burst time = 6

P3, arrival time = 2, burst time = 2

P4, arrival time = 2 (arrives after P3), burst time = 1

If a process arrives at time 0, requests a time slice of 1 unit, and is executed immediately, it will complete at time 1. Use FCFS for all tiebreaks.

Question 23:

What is the completion time for all four processes under First Come First Serve (FCFS)?

- A. P1:4, P2:9, P3:11, P4:12
- B. P1:4, P2:13, P3:6, P4:7
- C. P1:4, P2:10, P3:12, P4:13
- D. P1:4, P2:10, P3:11, P4:13
- E. P1:4, P2:10, P3:13, P4:11

Question 24:

What is the completion time for all four processes under Shortest Job First?

- A. P1:4, P2:13, P3:7, P4:5
- B. P1:4, P2:10, P3:13, P4:11
- C. P1:4, P2:13, P3:6, P4:7
- D. P1:7, P2:13, P3:5, P4:3
- E. P1:4, P2:10, P3:12, P4:13

Question 25:

What is the completion time for all four processes under Shortest Remaining Time First (a.k.a Shortest Job-to-Completion First)?

- A. P1:7, P2:13, P3:5, P4:3
- B. P1:5, P2:13, P3:7, P4:4
- C. P1:6, P2:13, P3:7, P4:4
- D. P1:5, P2:13, P3:7, P4:3
- E. P1:7, P2:13, P3:6, P4:3

Question 26:

What is the completion time for all four processes under Round Robin with time slice of 3?

- A. P1:13, P2:10, P3:7, P4:8
- B. P1:10, P2:8, P3:12, P4:13
- C. P1:10, P2:13, P3:8, P4:6
- D. P1:10, P2:13, P3:8, P4:9
- E. P1:13, P2:9, P3:6, P4:10

Question 27:

Which of the methods above give the shortest average turnaround time?

- A. Shortest Remaining Time First (SRTF)
- B. Shortest Job First (SJF)
- C. SJF and SRTF
- D. Round Robin (RR)
- E. First Come First Serve (FCFS)

Question 28:

You call malloc() in a C program to allocate a single byte of memory. Consider the following possible actions that may occur during this process: (1) sbrk() requests additional memory; (2) mmap() requests memory; (3) A physical page is allocated; (4) The page replacement algorithm is run. Which of the following sequences best represents the order of operations triggered by this malloc() call?

- A. (2) -> (3) -> (4)
- B. (1) -> (4) -> (3)
- C. (1) -> (3) -> (4)
- D. (1)
- E. (2) -> (3)

Question 29:

What will be the output of the program below?

```
int main() {  
    if (fork() || fork())  
        fork()  
    print("1");  
}
```

- A. 1111
- B. 11
- C. 111111
- D. 11111
- E. 111

Question 30:

An operating system implements a scheduling policy where each scheduled process runs for a maximum of 10 milliseconds unless it blocks or terminates earlier. After this time slice, the OS selects the next ready process in a round-robin fashion. Which of the following statements is correct about this scheduling policy?

- A. Processes with shorter CPU bursts are guaranteed to complete before processes with longer CPU bursts.
- B. The scheduler can cause an involuntary context switch to occur when a process's allotted timeslice expires.
- C. This is a non-preemptive scheduling policy, as processes voluntarily yield the CPU at the end of their timeslice.
- D. The policy can be implemented effectively even without hardware support.
- E. Decreasing the time quantum (timeslice) generally reduces context-switch overhead.

Question 31:

[Bonus Question] What is the class average on this test, excluding this bonus question?

- A. <60%
- B. 70-80%
- C. 80-90%
- D. 90-100%
- E. 60-70%